Practical No 1(AI)

Identify and Implement heuristic and search strategy for Travelling Salesperson Problem

```
import math

import random

……................................................

def distance(city1, city2):

    return math.sqrt((city1[0] - city2[0])**2 + (city1[1] - city2[1])**2)

……..................................................................


def total_distance(path, cities):

    cost = 0

    for i in range(len(path) - 1):

        cost += distance(cities[path[i]], cities[path[i + 1]])

    cost += distance(cities[path[-1]], cities[path[0]])  # Return to start

    return cost

……..................................................................

def nearest_neighbor(cities):

    n = len(cities)

    unvisited = list(range(n))

    path = [unvisited.pop(0)]


    while unvisited:

        last = path[-1]

        next_city = min(unvisited, key=lambda city: distance(cities[last],
cities[city]))
```

```
        path.append(next_city)
        unvisited.remove(next_city)

    return path
```

……

```
def hill_climbing(cities, path):
    improved = True
    best_path = path[:]
    best_cost = total_distance(best_path, cities)

    while improved:
        improved = False
        for i in range(1, len(path) - 1):
            for j in range(i + 1, len(path)):
                new_path = best_path[:]
                new_path[i:j] = reversed(new_path[i:j])
                new_cost = total_distance(new_path, cities)

                if new_cost < best_cost:
                    best_path = new_path
                    best_cost = new_cost
                    improved = True
    return best_path
```

……

```
cities = {
    0: (0, 0),
    1: (2, 6),
    2: (5, 3),
    3: (6, 7),
    4: (8, 2)
}
```

……

```
initial_path = nearest_neighbor(cities)
optimized_path = hill_climbing(cities, initial_path)

print("Initial Path (Nearest Neighbor):", initial_path)
print("Initial Distance:", total_distance(initial_path, cities))

print("\nOptimized Path (Hill Climbing):", optimized_path)
print("Optimized Distance:", total_distance(optimized_path, cities))
```

Home | Untitled20

localhost:8888/notebooks/Untitled20.ipynb

jupyter Untitled20 Last Checkpoint: 42 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[15]:
```
cities = {
    0: (0, 0),
    1: (2, 6),
    2: (5, 3),
    3: (6, 7),
    4: (8, 2)
}
```

[16]:
```
initial_path = nearest_neighbor(cities)
optimized_path = hill_climbing(cities, initial_path)

print("Initial Path (Nearest Neighbor):", initial_path)
print("Initial Distance:", total_distance(initial_path, cities))

print("\nOptimized Path (Hill Climbing):", optimized_path)
print("Optimized Distance:", total_distance(optimized_path, cities))
```

```
Initial Path (Nearest Neighbor): [0, 2, 4, 3, 1]
Initial Distance: 24.826055308102607

Optimized Path (Hill Climbing): [0, 2, 4, 3, 1]
Optimized Distance: 24.826055308102607
```

[ ]:

Type here to search

Disclosure Day officia... ENG 04:42 20-03-2026